

09 · Built-in Functions

Why this matters in interviews. Oracle interviews love function questions: *NVL* vs *COALESCE*, *SUBSTR* / *INSTR* puzzles, "add six months to today", *TO_CHAR* date formats, *ROUND* with a negative number, *REGEXP_LIKE*. They are quick to answer if you know the families.

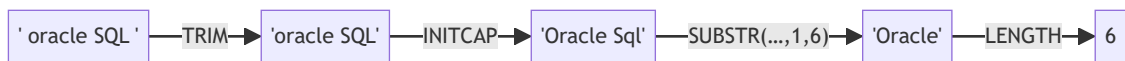
In this chapter: single-row vs aggregate functions · character functions · numeric functions · date and time functions · conversion functions (*TO_CHAR*, *TO_DATE*, ...) · NULL-handling functions · regular-expression functions.

Convention. Every "today" in this chapter means **16-AUG-2025** (the day the examples were prepared), so *SYSDATE* gives that date. *DUAL* is Oracle's one-row dummy table (Chapter 01), used to call a function without a real table.

9.1 Single-row vs aggregate functions

	Single-row functions	Aggregate functions
Called on	each row	a set of rows
Returns	one value per row	one value per group
Examples	UPPER, SUBSTR, ROUND, TO_CHAR, NVL	SUM, AVG, COUNT, MIN, MAX (Chapter 06)
Usable in	SELECT, WHERE, ORDER BY, GROUP BY ...	SELECT, HAVING, ORDER BY (not WHERE)

Single-row functions can be **nested** — the innermost runs first: *UPPER*(*SUBSTR*(*emp_name*, 1, 3)).



9.2 Character (string) functions

Case conversion

Function	Description	Example	Result
UPPER(str)	converts to uppercase	UPPER('oracle')	ORACLE
LOWER(str)	converts to lowercase	LOWER('ORACLE')	oracle
INITCAP(str)	first letter of each word uppercase, the rest lowercase	INITCAP('hello world')	Hello World

Character manipulation

Function	Description	Example	Result
CONCAT(a, b)	joins two strings (two arguments only in Oracle)	CONCAT('Data', 'Base')	DataBase
a b	concatenation operator — chain as many as you like	'Data' 'Base' '!'	DataBase!
LPAD(str, len, pad)	pads on the left to total length <i>len</i>	LPAD('123', 5, '0')	00123
RPAD(str, len, pad)	pads on the right	RPAD('123', 5, '*')	123**
LTRIM(str, chars)	removes leading characters	LTRIM('000123', '0')	123
RTRIM(str, chars)	removes trailing characters	RTRIM('123000', '0')	123
TRIM([LEADING TRAILING BOTH] ch FROM str)	trims from the left and right (default: spaces, both)	TRIM(' hi ')	hi

LPAD / *RPAD* **truncate** if the string is already longer than *len*. *LTRIM* / *RTRIM* take a **set of characters**, not a substring: *LTRIM*('xyxyhello', 'xy') removes any leading *x*'s and *y*'s → *hello*.

Substring and length

Function	Description	Example	Result
LENGTH(str)	number of characters	LENGTH('Oracle')	6
SUBSTR(str, start, len)	part of a string from <i>start</i> (1-based), <i>len</i> chars	SUBSTR('Database', 2, 3)	ata
INSTR(str, sub, start, occurrence)	position of <i>sub</i> (0 if absent)	INSTR('Database', 'a')	2
REPLACE(str, search, replace)	replaces every occurrence	REPLACE('2025-08-16', '-', '/')	2025/08/16
TRANSLATE(str, from, to)	replaces characters one-for-one	TRANSLATE('12345', '123', 'ABC')	ABC45

```

SELECT UPPER('oracle')           AS up,
       LOWER('ORACLE')           AS lo,
       INITCAP('hello world')    AS ic,
       LENGTH('Oracle')          AS len,
       SUBSTR('Database', 2, 3)   AS sub,
       SUBSTR('Database', -4)     AS last_four,
       INSTR('Database', 'a')     AS first_a,
       INSTR('Database', 'a', 1, 2) AS second_a,
       REPLACE('2025-08-16', '-', '/') AS rep,
       TRANSLATE('12345', '123', 'ABC') AS tr
FROM dual;

```

Result

up	lo	ic	len	sub	last_four	first_a	second_a	rep	tr
ORACLE	oracle	Hello World	6	ata	base	2	4	2025/08/16	ABC45

Notes worth knowing:

- **SUBSTR with a negative start counts from the end:** `SUBSTR('Database', -4) = base`. A start of `0` is treated as `1`.
- **INSTR returns 0, not an error, when nothing is found;** the 4th argument finds the *n*-th occurrence.
- **REPLACE** with no third argument **removes** the search text. **TRANSLATE** maps characters **position by position**; characters in `from` that have no counterpart in `to` are **removed** — `TRANSLATE('12345', '123', 'A')` gives `A45`. (You cannot pass an empty `to` string: Oracle would return NULL.)

```

SELECT LPAD('123', 5, '0') AS lpad,
       RPAD('123', 5, '*') AS rpad,
       LTRIM('000123', '0') AS ltrim,
       RTRIM('123000', '0') AS rtrim,
       CONCAT('Data', 'Base') AS concat
FROM dual;

```

Result

lpad	rpad	ltrim	rtrim	concat
00123	123**	123	123	DataBase

Formatting and encoding

Function	Description	Example	Result
<code>CHR(n)</code>	the character with ASCII code <i>n</i>	<code>CHR(65)</code>	A
<code>ASCII(ch)</code>	the ASCII code of a character	<code>ASCII('A')</code>	65
<code>SOUNDEX(str)</code>	phonetic code — matches words that sound alike	<code>SOUNDEX('Smith') = SOUNDEX('Smyth')</code>	TRUE (both S530)

```

SELECT CHR(65) AS chr, ASCII('A') AS ascii, SOUNDEX('Smith') AS s1, SOUNDEX('Smyth') AS s2 FROM dual;

```

Result

chr	ascii	s1	s2
A	65	S530	S530

Byte-based variants

`LENGTHB`, `SUBSTRB`, `INSTRB` count **bytes** instead of characters (they differ only for multi-byte character sets such as Unicode). `NANVL(x, value)` returns `value` if `x` is NaN (not-a-number) — relevant for `BINARY_FLOAT` / `BINARY_DOUBLE`.

Practical string recipes

```

SELECT emp_name,
       UPPER(SUBSTR(emp_name, 1, 3)) AS code,
       LPAD(TO_CHAR(emp_id), 6, '0') AS padded_id,
       emp_name || ' — ' || job      AS label
FROM emp
WHERE dept_id = 30
ORDER BY emp_id;

```

Result

emp_name	code	padded_id	label
Farhan	FAR	000106	Farhan — Manager
Gita	GIT	000107	Gita — Sales Rep
Hari	HAR	000108	Hari — Sales Rep

Case-insensitive search (Oracle string comparison is case-sensitive):

```
SELECT * FROM emp WHERE UPPER(emp_name) = 'ASHA';
SELECT * FROM emp WHERE UPPER(emp_name) LIKE 'A%';
```

Username from an email address — SUBSTR up to the position of @:

```
SELECT 'kavya.r@example.com' AS email,
       LENGTH('kavya.r@example.com') AS email_length,
       SUBSTR('kavya.r@example.com', 1, INSTR('kavya.r@example.com', '@') - 1) AS username
FROM dual;
```

Result

email	email_length	username
kavya.r@example.com	19	kavya.r

Last four digits of a phone number, from a 10-digit mobile : SUBSTR(mobile, 7) — or the middle four: SUBSTR(mobile, 3, 4) .

Trap — concatenation and NULL. In Oracle 'abc' || NULL is 'abc' (NULL is treated as an empty string). In MySQL, PostgreSQL and SQL Server the result is NULL. Also CONCAT('a', NULL) is 'a' in Oracle.

9.3 Numeric functions

ROUND(number, integer)

Syntax: ROUND(number, integer) — number is the value to round; integer is optional and says **how many digits to keep**.

Positive integer — round to that many **decimal places**:

```
SELECT ROUND(123.456, 2) AS two_places,    -- 123.46
       ROUND(123.456, 1) AS one_place,     -- 123.5
       ROUND(123.456, 0) AS zero_places    -- 123
FROM dual;
```

Result

two_places	one_place	zero_places
123.46	123.5	123

Negative integer — round to the **left of the decimal point** (tens, hundreds, thousands):

```
SELECT ROUND(12345, -1) AS to_ten,        -- 12350
       ROUND(12345, -2) AS to_hundred,    -- 12300
       ROUND(12345, -3) AS to_thousand    -- 12000
FROM dual;
```

Result

to_ten	to_hundred	to_thousand
12350	12300	12000

Integer omitted — it defaults to 0 (round to a whole number):

```
SELECT ROUND(15.67) AS a, ROUND(15.23) AS b, ROUND(2.5) AS half_up, ROUND(-2.5) AS half_down FROM dual;
```

Result

a	b	half_up	half_down
16	15	3	-3

Ties are rounded **away from zero** (2.5 → 3 , -2.5 → -3) — not "banker's rounding".

The other everyday numeric functions

Function	Description	Example	Result
TRUNC(n, d)	cuts off (never rounds) to d decimals	TRUNC(123.456, 1)	123.4
CEIL(n)	smallest integer ≥ n	CEIL(7.1)	8
FLOOR(n)	largest integer ≤ n	FLOOR(7.9)	7
MOD(a, b)	remainder (Oracle's %)	MOD(10, 3)	1

Function	Description	Example	Result
ABS(n)	absolute value	ABS(-7)	7
POWER(a, b)	a to the power b	POWER(2, 10)	1024
SQRT(n)	square root	SQRT(144)	12
SIGN(n)	-1, 0 or 1	SIGN(-8)	-1

```
SELECT TRUNC(123.456, 1) AS trunc, CEIL(7.1) AS ceil, FLOOR(7.9) AS floor,
       MOD(10, 3) AS mod, MOD(-10, 3) AS mod_neg, POWER(2, 10) AS power, SIGN(-8) AS sign
FROM dual;
```

Result

trunc	ceil	floor	mod	mod_neg	power	sign
123.4	8	7	1	-1	1024	-1

MOD(-10, 3) is -1 in Oracle (the result takes the sign of the **dividend**). **Even/odd**: MOD(n, 2) = 0 is even.

9.4 Date and time functions

The types and "now" functions

Type	Stores
DATE	date and time (to the second)
TIMESTAMP	a date plus fractional seconds
TIMESTAMP WITH TIME ZONE	timestamp and its time-zone offset
TIMESTAMP WITH LOCAL TIME ZONE	normalised to the DB's time zone, shown in the session's zone

Function	Returns
SYSDATE	current date and time from the database server (e.g. 16-AUG-25 10:45:32)
CURRENT_DATE	current date and time in the session's time zone
CURRENT_TIMESTAMP	current timestamp with fractional seconds and time zone
LOCALTIMESTAMP	current timestamp without a time zone
SYSTIMESTAMP	current timestamp from the database host machine , including time zone

SELECT SYSDATE FROM dual; — the display uses NLS_DATE_FORMAT (default DD-MON-RR), which **hides the time**; use TO_CHAR to see it.

Date arithmetic

- date + n / date - n add or subtract **days** (fractions allowed: + 1/24 = one hour).
- date1 - date2 is a **number of days**.

```
SELECT first_name, hire_date FROM employees WHERE hire_date > SYSDATE - 30; -- hired in the last 30 days
```

Functions

Function	Description	Example (today = 16-AUG-2025)	Result
ADD_MONTHS(d, n)	add (or subtract) months	ADD_MONTHS(SYSDATE, 6)	16-FEB-2026
		ADD_MONTHS(SYSDATE, -3)	16-MAY-2025
MONTHS_BETWEEN(d1, d2)	months between two dates (fractional)	MONTHS_BETWEEN(SYSDATE, DATE '2025-01-01')	7.4839...
NEXT_DAY(d, 'DAY')	the next given weekday after d	NEXT_DAY(SYSDATE, 'MONDAY')	18-AUG-2025
LAST_DAY(d)	the last day of the month	LAST_DAY(SYSDATE)	31-AUG-2025
ROUND(d, fmt)	rounds a date to the nearest unit	ROUND(SYSDATE, 'MONTH')	01-SEP-2025 (day > 15 rounds up)
		ROUND(SYSDATE, 'YEAR')	01-JAN-2026
TRUNC(d, fmt)	truncates a date down to the unit	TRUNC(SYSDATE, 'MONTH')	01-AUG-2025 (first day of the month)
		TRUNC(SYSDATE, 'YEAR')	01-JAN-2025
		TRUNC(SYSDATE)	16-AUG-2025 00:00:00 (removes the time)
EXTRACT(part FROM d)	pulls out YEAR, MONTH, DAY (and HOUR ... for timestamps)	EXTRACT(YEAR FROM SYSDATE)	2025

```

SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY')           AS today,
       TO_CHAR(ADD_MONTHS(SYSDATE, 6), 'DD-MON-YYYY') AS plus_6_months,
       TO_CHAR(ADD_MONTHS(SYSDATE, -3), 'DD-MON-YYYY') AS minus_3_months,
       ROUND(MONTHS_BETWEEN(SYSDATE, DATE '2025-01-01'), 2) AS months_since_new_year,
       TO_CHAR(NEXT_DAY(SYSDATE, 'MONDAY'), 'DD-MON-YYYY') AS next_monday,
       TO_CHAR(NEXT_DAY(SYSDATE, 'FRIDAY'), 'DD-MON-YYYY') AS next_friday,
       TO_CHAR(LAST_DAY(SYSDATE), 'DD-MON-YYYY') AS month_end,
       TO_CHAR(TRUNC(SYSDATE, 'MONTH'), 'DD-MON-YYYY') AS month_start,
       TO_CHAR(ROUND(SYSDATE, 'MONTH'), 'DD-MON-YYYY') AS rounded_month,
       TO_CHAR(TRUNC(SYSDATE, 'YEAR'), 'DD-MON-YYYY') AS year_start,
       TO_CHAR(ROUND(SYSDATE, 'YEAR'), 'DD-MON-YYYY') AS rounded_year,
       EXTRACT(YEAR FROM SYSDATE) AS yr,
       EXTRACT(MONTH FROM SYSDATE) AS mon,
       EXTRACT(DAY FROM SYSDATE) AS dy
FROM dual;

```

Result

today	plus_6_months	minus_3_months	months_since_new_year	next_monday	next_friday	month_end	month_start	rounded_month	year_start	rounded_year	yr	n
16-AUG-2025	16-FEB-2026	16-MAY-2025	7.48	18-AUG-2025	22-AUG-2025	31-AUG-2025	01-AUG-2025	01-SEP-2025	01-JAN-2025	01-JAN-2026	2025	8

`NEXT_DAY(d, 'MONDAY')` is **strictly after** `d` — if today is Monday, it returns *next* Monday. The day-name language depends on the session's `NLS_DATE_LANGUAGE`.

Practical date recipes

Experience in years (rounded to one decimal), and **age from date of birth** (whole years):

```

SELECT emp_name, hire_date,
       ROUND(MONTHS_BETWEEN(SYSDATE, hire_date) / 12, 1) AS years_experience,
       TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date) / 12) AS whole_years
FROM emp
WHERE dept_id = 30
ORDER BY emp_id;

```

Result

emp_name	hire_date	years_experience	whole_years
Farhan	2017-05-30	8.2	8
Gita	2021-01-12	4.6	4
Hari	2022-06-18	3.2	3

`TRUNC(MONTHS_BETWEEN(SYSDATE, dob) / 12)` is the standard **age** calculation.

Day, month and year of the hire date as separate columns:

```

SELECT emp_name,
       EXTRACT(DAY FROM hire_date) AS day,
       EXTRACT(MONTH FROM hire_date) AS month,
       EXTRACT(YEAR FROM hire_date) AS year
FROM emp
WHERE emp_id <= 103
ORDER BY emp_id;

```

Result

emp_name	day	month	year
Asha	10	1	2015
Bala	15	3	2016
Chitra	1	7	2018

Employees hired before 2018, oldest first:

```

SELECT emp_name, hire_date FROM emp WHERE EXTRACT(YEAR FROM hire_date) < 2018 ORDER BY hire_date;

```

Result

emp_name	hire_date
Asha	2015-01-10
Bala	2016-03-15

emp_name	hire_date
Farhan	2017-05-30

9.5 Conversion functions

Oracle converts types **implicitly** when it can (`'100' + 5 = 105`, a date string compared with a `DATE`) — but relying on it is fragile: it depends on `NLS` settings and blocks index use. Convert **explicitly**:

Function	Converts	Example
<code>TO_CHAR(date, fmt)</code>	date → string	<code>TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS')</code>
<code>TO_CHAR(number, fmt)</code>	number → string	<code>TO_CHAR(1234.5, '9,999.99')</code>
<code>TO_DATE(string, fmt)</code>	string → DATE	<code>TO_DATE('16-AUG-2025', 'DD-MON-YYYY')</code>
<code>TO_TIMESTAMP(string, fmt)</code>	string → TIMESTAMP	<code>TO_TIMESTAMP('2025-08-16 10:30:45', 'YYYY-MM-DD HH24:MI:SS')</code>
<code>TO_NUMBER(string, fmt)</code>	string → number	<code>TO_NUMBER('1,234.50', '9,999.99') → 1234.5</code>
<code>CAST(x AS type)</code>	standard conversion	<code>CAST('42' AS NUMBER)</code>

Date format elements

Sample: `16-AUG-2025 10:45:32` (a Saturday).

Element	Meaning	Gives
YYYY / YY	4- / 2-digit year	2025 / 25
MM	month number	08
MON	abbreviated month	AUG
MONTH	full month name (blank-padded to 9 characters)	AUGUST
DD	day of month	16
DDD	day of year	228
DAY	full weekday name (blank-padded to 9)	SATURDAY
DY	abbreviated weekday	SAT
D	weekday number (1–7, depends on NLS)	7
HH / HH12 / HH24	hour (12h / 12h / 24h)	10
MI	minute	45
SS	second	32
AM / PM	meridian indicator	AM
Q	quarter	3
WW	week of year	33
FF	fractional seconds (timestamps)	
FM	fill mode — suppresses padding and leading zeros	
TH, SP	ordinal / spelled-out (<code>DDTH</code> → <code>16TH</code>)	

Case follows the format: `Month` → `August`, `MONTH` → `AUGUST`, `month` → `august`.

```
SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY')           AS a,
       TO_CHAR(SYSDATE, 'Day, Month DD YYYY')     AS b_padded,
       TO_CHAR(SYSDATE, 'FMDay, FMMonth DD, YYYY') AS c_trimmed
FROM dual;
```

Result

a	b_padded	c_trimmed
16-AUG-2025	Saturday , August 16 2025	Saturday, August 16, 2025

The padded version (`b_padded`) has **stray spaces** — `Day` is padded to nine characters (`Saturday` has one trailing space) and `Month` to nine (`August` has three), so you get `Saturday , August 16 2025`. Add `FM` (`c_trimmed`) to remove the padding. This matters whenever you build messages or file names from dates.

The "quote inside a string" message

A very common exercise: *"Dear Elavarasi, you were born on '23, Monday - March - 1970'."* — the **date must be wrapped in literal single quotes** inside the output. To put a `'` inside a string literal you **double it**:

```
SELECT 'Dear ' || emp_name ||
       ', you were born on ' || TO_CHAR(dob, 'DD, Day - Month - YYYY') || ' ' AS message
FROM   employee;
```

Piece	What it produces
' , you were born on ' '	the text , you were born on ' — the string ends with a doubled ' ' = one literal quote
TO_CHAR(dob, 'DD, Day - Month - YYYY')	23, Monday - March - 1970 (with Day / Month padding — use FMDay / FMMonth to trim)
' ' '	four quotes = the string delimiter pair around one doubled quote → a single literal ' ' closing the date

Why four? An outer '...' string containing one doubled ' ': ' (open) + ' ' (one literal quote) + ' ' (close) = ' ' '. Oracle's alternative quoting avoids the doubling: q'[it's fine]'.

Number format masks

Mask	Value	Result	Notes
'9,999.99'	1234.5	1,234.50	9 = digit, position kept only if needed; leading space is the sign
'FM9,999.99'	1234.5	1,234.50	FM drops the leading space
'0000'	7	0007	0 forces leading zeros
'FM000'	7	007	
'L99,999'	12345	\$12,345	L = local currency symbol (taken from NLS_CURRENCY; \$, ₹, € ...)
'9999'	123456	#####	too many digits for the mask ⇒ hashes

9.6 NULL-handling functions

Function	Behaviour	Example	Result
NVL(a, b) (Oracle)	b if a is NULL, else a	NVL(NULL, 100)	100
NVL2(a, b, c) (Oracle)	b if a is not NULL, else c	NVL2('X', 'Y', 'Z')	Y
NULLIF(a, b)	NULL if a = b, else a	NULLIF(10, 10)	NULL
COALESCE(a, b, c, ...) (ANSI)	the first non-NULL argument	COALESCE(NULL, NULL, 'Hello', 'World')	Hello

```
SELECT emp_name, salary, comm,
       NVL(comm, 0) AS comm_or_zero,
       salary + NVL(comm, 0) AS total_pay,
       NVL2(comm, 'earns commission', 'salary only') AS pay_type
FROM   emp
WHERE  dept_id = 30
ORDER  BY emp_id;
```

Result

emp_name	salary	comm	comm_or_zero	total_pay	pay_type
Farhan	65000	NULL	0	65000	salary only
Gita	35000	5000	5000	40000	earns commission
Hari	35000	3000	3000	38000	earns commission

NULLIF — avoid division by zero, compare two columns

```
SELECT salary, bonus, salary / NULLIF(bonus, 0) AS salary_per_bonus FROM employees;
```

If bonus = 0 then NULLIF(bonus, 0) returns NULL, and dividing by NULL gives **NULL instead of an error** (ORA-01476: divisor is equal to zero). Comparing two columns: NULLIF(salary, bonus) is NULL when they are equal and salary otherwise.

COALESCE — the first non-NULL, in order of preference

```
SELECT COALESCE(NULL, NULL, 'Hello', 'World') AS first_non_null,
       NVL(NULL, 100) AS nvl_result,
       NULLIF(10, 10) AS nullif_equal,
       NULLIF(10, 20) AS nullif_different
FROM   dual;
```

Result

first_non_null	nvl_result	nullif_equal	nullif_different
Hello	100	NULL	10

Typical use — contact-number fallback: `COALESCE(mobile, home_phone, office_phone, 'no number')`.

NVL VS COALESCE

	NVL	COALESCE
Standard	Oracle only	ANSI (works everywhere)
Arguments	exactly 2	2 or more
Evaluation	evaluates both arguments	short-circuits at the first non-NULL
Type handling	converts the 2nd argument to the 1st's type	all arguments must be compatible

`ISNULL(a, b)` (SQL Server) and `IFNULL(a, b)` (MySQL) are the `NVL` equivalents there. For general branching use `CASE` (Chapter 04).

9.7 Regular-expression functions (Oracle)

`LIKE` handles simple patterns. For real pattern matching Oracle offers **POSIX-style regular expressions**:

Function	Purpose
<code>REGEXP_LIKE(col, pattern [, match_option])</code>	condition — does the value match? (like <code>LIKE</code> with regex power)
<code>REGEXP_INSTR(col, pattern, ...)</code>	position of the match (0 if none)
<code>REGEXP_SUBSTR(col, pattern, ...)</code>	extracts the matching text
<code>REGEXP_REPLACE(col, pattern, replacement, ...)</code>	replaces matching text
<code>REGEXP_COUNT(col, pattern, ...)</code> (<i>11g+</i>)	counts matches

The pattern building blocks

Pattern	Meaning	Example
<code>^</code>	start of string	<code>^A</code> → starts with A
<code>\$</code>	end of string	<code>son\$</code> → ends with "son"
<code>.</code>	any single character	<code>h.t</code> → hat, hit, hot
<code>[abc]</code>	any of a, b, c	<code>J[ao]n</code> → Jan, Jon
<code>[^abc]</code>	not a, b or c	<code>[^0-9]</code> → not a digit
<code>[0-9]</code>	any digit	
<code>+</code>	one or more	<code>[0-9]+</code>
<code>*</code>	zero or more	<code>a*</code>
<code>{n}</code>	exactly <i>n</i>	<code>[0-9]{3}</code>
<code>{n,}</code>	at least <i>n</i>	<code>[0-9]{2,}</code>
<code>{n,m}</code>	between <i>n</i> and <i>m</i>	<code>[0-9]{2,4}</code>
<code>\ </code>	alternation (OR)	<code>cat\ dog</code>
<code>()</code>	grouping	<code>(ab)+</code>

`match_option` (last argument): `'i'` case-insensitive · `'c'` case-sensitive · `'n'` matches newline · `'m'` multi-line · `'x'` ignore whitespace.

```
SELECT emp_name FROM emp WHERE REGEXP_LIKE(emp_name, '^[AE]') ORDER BY emp_name;
```

Result

emp_name
Asha
Esha

Names starting with A or E** — the Oracle-safe replacement for `LIKE '^[AE]'`.

```
SELECT REGEXP_INSTR('oracle123sql', '[0-9]+')      AS pos,          -- where the first digits start
       REGEXP_SUBSTR('abc123xyz', '[0-9]+')        AS digits,      -- the digits themselves
       REGEXP_REPLACE('abc123xyz', '[0-9]+', '###') AS masked,
       REGEXP_COUNT('a1b2c3', '[0-9]')            AS how_many_digits
FROM dual;
```

Result

pos	digits	masked	how_many_digits
7	123	abc###xyz	3

`REGEXP_INSTR('oracle123sql', '[0-9]+')` is **7** — the first digit sits at position 7. Note `REGEXP_LIKE` is a *condition* usable directly in `WHERE`, not a value you `SELECT`.

Validation with `CHECK` constraints — a department name must be **one capitalised word**, a project domain must **not** be all digits:

```
CONSTRAINT chk_name CHECK (REGEXP_LIKE(dept_name, '^[A-Z][a-z]+$')),
CONSTRAINT chk_loc CHECK (REGEXP_LIKE(dept_loc, '^[A-Z]+$')),
CONSTRAINT chk_domain CHECK (NOT REGEXP_LIKE(project_domain, '[0-9]+$'))
```

```
SELECT CASE WHEN REGEXP_LIKE('Sales', '^[A-Z][a-z]+$') THEN 'valid' ELSE 'invalid' END AS proper_word,
CASE WHEN REGEXP_LIKE('sales', '^[A-Z][a-z]+$') THEN 'valid' ELSE 'invalid' END AS lowercase_start,
CASE WHEN REGEXP_LIKE('12345', '[0-9]+$') THEN 'digits only' ELSE 'other' END AS all_digits
FROM dual;
```

Result

proper_word	lowercase_start	all_digits
valid	invalid	digits only

`REGEXP_LIKE` is a **condition**, not a value — you cannot put it directly in a `SELECT` list in Oracle SQL. Use it in `WHERE`, `CHECK`, or inside `CASE WHEN ...` as above.

Interview corner

- Q. `NVL` vs `COALESCE` ?** `NVL(a, b)` is Oracle-specific with two arguments; `COALESCE` is ANSI, takes many arguments and short-circuits.
- Q. What does `NVL2` do?** `NVL2(expr, if_not_null, if_null)` — it tests whether `expr` is NULL and returns one of the other two.
- Q. `NULLIF` — why is it useful?** It returns NULL when the two values are equal — the classic use is `x / NULLIF(y, 0)` to avoid divide-by-zero.
- Q. How do you add six months to today? Find the last day of the month?** `ADD_MONTHS(SYSDATE, 6) · LAST_DAY(SYSDATE)`.
- Q. What does `ROUND(SYSDATE, 'MONTH')` return?** The first day of the *nearest* month — the next month if today's day is after the 15th.
- Q. `ROUND(12345, -2)` ?** `12300` — a negative digit count rounds to the left of the decimal point.
- Q. `SUBSTR` vs `INSTR` ?** `SUBSTR` **extracts** part of a string; `INSTR` **finds the position** of a substring (0 if absent). Combine them to slice around a delimiter.
- Q. How do you display a date as *Saturday, August 16, 2025* ?** `TO_CHAR(d, 'FMDay, FMMonth DD, YYYY')` — `FM` removes the blank padding.
- Q. Age from date of birth?** `TRUNC(MONTHS_BETWEEN(SYSDATE, dob) / 12)`.
- Q. `'abc' || NULL` in Oracle?** `'abc'` — NULL acts as an empty string in concatenation.
- Q. `LIKE '[AR]'` doesn't work in Oracle — alternatives?** `REGEXP_LIKE(col, '^[AR]')`, or `LIKE 'A%' OR LIKE 'R%'`.

Quick recap

- **Single-row** functions run per row; **aggregates** per group. Nest them; `DUAL` for standalone calls.
- **Strings:** `UPPER` `LOWER` `INITCAP` · `CONCAT` / `||` · `LPAD` `RPAD` `LTRIM` `RTRIM` `TRIM` · `LENGTH` `SUBSTR` `INSTR` `REPLACE` `TRANSLATE` · `CHR` `ASCII` `SOUNDEX`. `SUBSTR` takes negative starts; `INSTR` gives 0 for "not found".
- **Numbers:** `ROUND(n, d)` (negative `d` rounds left of the point) · `TRUNC` `CEIL` `FLOOR` `MOD` `ABS` `POWER` `SQRT` `SIGN`.
- **Dates:** `SYSDATE`, +/- days, `ADD_MONTHS`, `MONTHS_BETWEEN`, `NEXT_DAY`, `LAST_DAY`, `ROUND/TRUNC(date, 'MONTH'|'YEAR')`, `EXTRACT`.
- **Conversion:** `TO_CHAR`, `TO_DATE`, `TO_NUMBER`, `TO_TIMESTAMP`; format `Day / Month pad` — use **FM**; a literal `'` in a string is written `''`.
- **NULLs:** `NVL`, `NVL2`, `NULLIF`, `COALESCE`.
- **Regex:** `REGEXP_LIKE` / `INSTR` / `SUBSTR` / `REPLACE` / `COUNT` — the fix for `[ ]` patterns that `LIKE` can't do in Oracle.